STA 3402 – Machine Learning & Neural Networks

Lecture 1 (Day 1) – Course Introduction & Review of Statistical Learning

Lecturer: Dr. Samir, Department of Statistics, University of Colombo

Complete study notes made from the lecture recording, the whiteboard and the R code (simple English)

Contents

1. Course information and class rules
2. Syllabus – what we will learn
3. AI, Machine Learning and Deep Learning
4. Supervised learning (regression & classification)
5. Unsupervised learning (clustering & recommender systems)
6. Steps to solve a data problem (house price example)
7. Data preprocessing: duplicates, dummy coding, missing values, outliers, scaling
8. Exploratory Data Analysis (EDA)
9. Train / test split
10. Models for a regression problem
11. Important words: predictors, response, multicollinearity
12. Linear regression, Ridge and Lasso (regularization)
13. Decision trees and Random Forest
14. Variable selection and the variable importance plot
15. Dimension reduction – PCA
16. Career advice from the lecturer
17. R practical – the mtcars example (full code explained line by line)
18. Assignment 1 – Kaggle competition
19. Summary and revision questions

1. Course information and class rules

Item	Details
Lecture time	Monday, 6.00 pm – 9.00 pm, online (Zoom). Some weeks the lecturer cannot come on Monday, so an extra class may be held on another day (e.g. **Wednesday at 6 pm** in the first week; the R file says Wed 7 pm – check the WhatsApp group).
Lab sessions	There will also be lab sessions. After the labs are reserved, the time will be announced. You go to the lab and do the lab sheet.
Credits	**4 credits**. In theory that means 4 hours per week, but because of lab sessions we will meet online about **2–3 hours per week**.
Assessment	There will be take-home assignments and in-class assignments. As per the syllabus (typical for a 4-credit subject), about **40% continuous assessment + 60% final exam**. (Students said that in the previous subject, Statistical Learning, there were no assignments because of the schedule – only a final exam with written and lab parts.)
Materials	Shared through a **Google Drive** link (the LMS was not yet available). Link is also in the WhatsApp group.
Software	Mainly **R / RStudio** (the class uses R). RStudio is installed in the lab computers.

Class rules

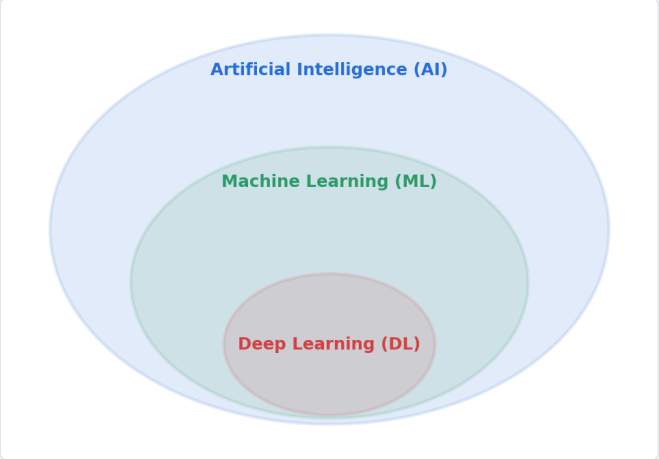
- **Attendance** is taken from your Zoom name. Rename yourself in the given format (registration number + name). If two students share one laptop, put both names in the screen name.
- To answer a question you can use the **thumbs-up** button, type in the **chat**, or unmute and speak.
- This is a very **practical** subject. Code is run during the lecture, so from the next week you should join from a **laptop or desktop**, not a phone.
- If you have any difficulty (for example, finishing at 9 pm is too late), tell the lecturer on WhatsApp.

2. Syllabus – what we will learn

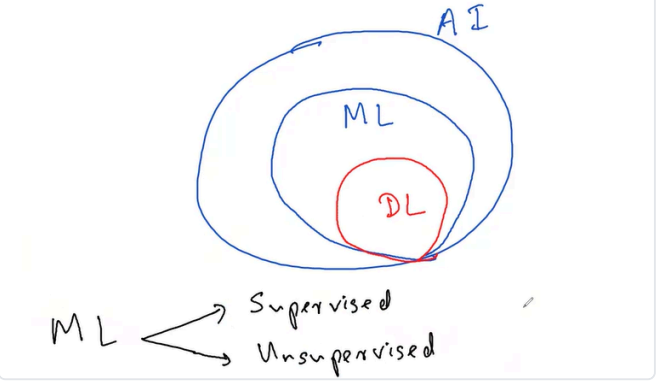
Main part	Topics
Neural networks (main topic, same as the module title)	Biological function of a neuron; neural network basics; back-propagation algorithm; associative learning; learning algorithms and applications in pattern recognition, (signal/image) processing and computer vision.
Supervised learning	Parametric algorithms; Support Vector Machines (SVM); kernels; neural networks.
Unsupervised learning	Clustering; dimension reduction; recommender systems.
Deep learning & reinforcement learning	Deep learning; reinforcement learning – the agent, the environment, related problems, and the Markov decision process.

Today's lecture is a **revision of Statistical Learning** (last semester), because we need it for the rest of the course.

3. AI, Machine Learning and Deep Learning



Clean diagram



Lecturer's whiteboard

- **Artificial Intelligence (AI)** – the biggest area: making computers do "intelligent" tasks.
- **Machine Learning (ML)** – a **subset** of AI. The computer **learns patterns from data** instead of being given fixed rules.
- **Deep Learning (DL)** – a **subset** of ML. It uses neural networks with many layers.

So: every DL method is ML, and every ML method is AI – but not the other way round.

Machine learning has **two main types**:

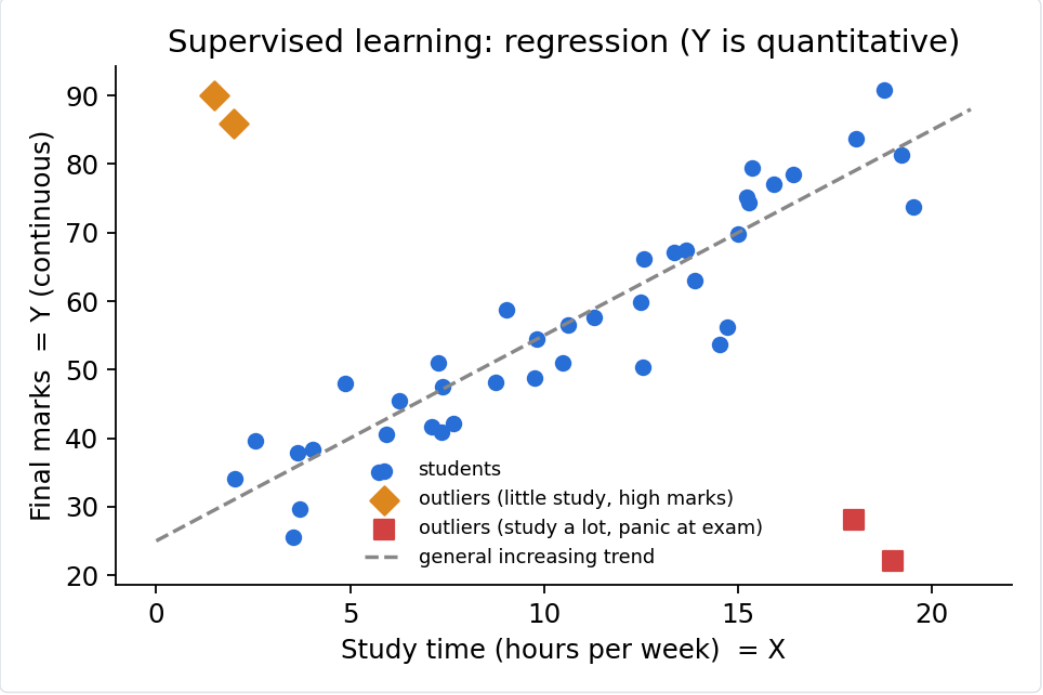
Supervised learning	Unsupervised learning
We have a **response (target / Y) variable**. The model learns to predict Y from the X variables.	There is **no response variable**. We only look for patterns or groups in the X variables.
Examples given by students: mobile phone price prediction, vegetable price models, regression models, classification trees.	Examples: clustering, recommender systems, dimension reduction.

4. Supervised learning

4.1 Example 1 – Study time and final marks (regression)

Suppose our data set has three columns: **Registration number (ID)**, **Study time** (hours per week spent on this course) and **Final marks**.

ID	Study time (hrs/week)	Marks
1	15	80
2	3	35
…	…	…



Study time vs marks – general increasing trend with a few outliers

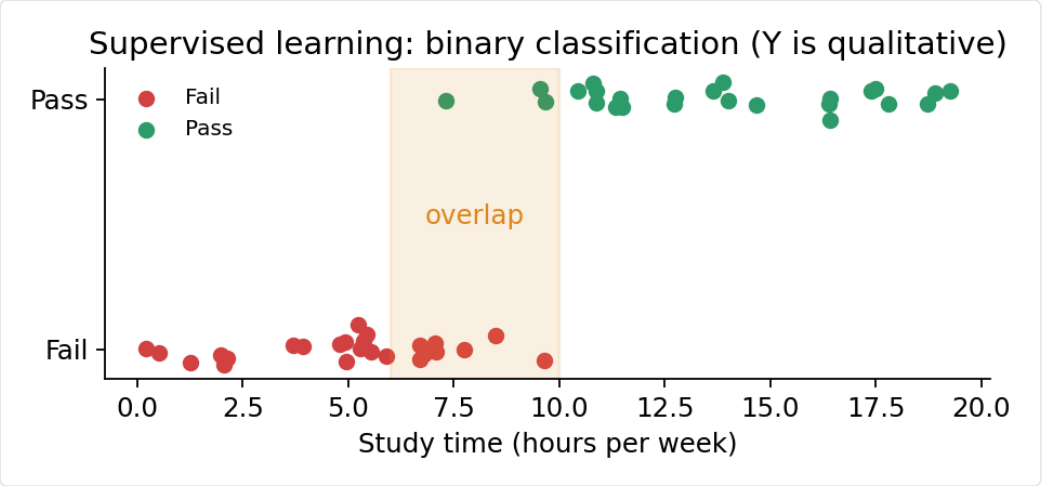
What we usually see when we plot the data:

- Hard-working students spend many hours (15–20 per week) and most of them get **good marks**.
- Students who are busy (part-time job, relationships, etc.) study less and get **low marks**.
- Average students study an average time and get **average marks**.

- A few students study only 1–2 hours before the exam but score very **high** marks, and a few study every day but **panic at the exam** and score low. These unusual points are called **outliers**.

In general there is an **increasing trend**. Here **marks is a continuous (quantitative) variable**. It is the **Y variable** = response = target = dependent variable. Because we have a Y variable, this is **supervised learning** (a regression problem).

4.2 Example 2 – Pass or Fail (classification)



Y has only two categories → binary classification

Now the response is the **grade: Pass or Fail**. Students who study little usually fail, students who study more usually pass, and there is some overlap in the middle.

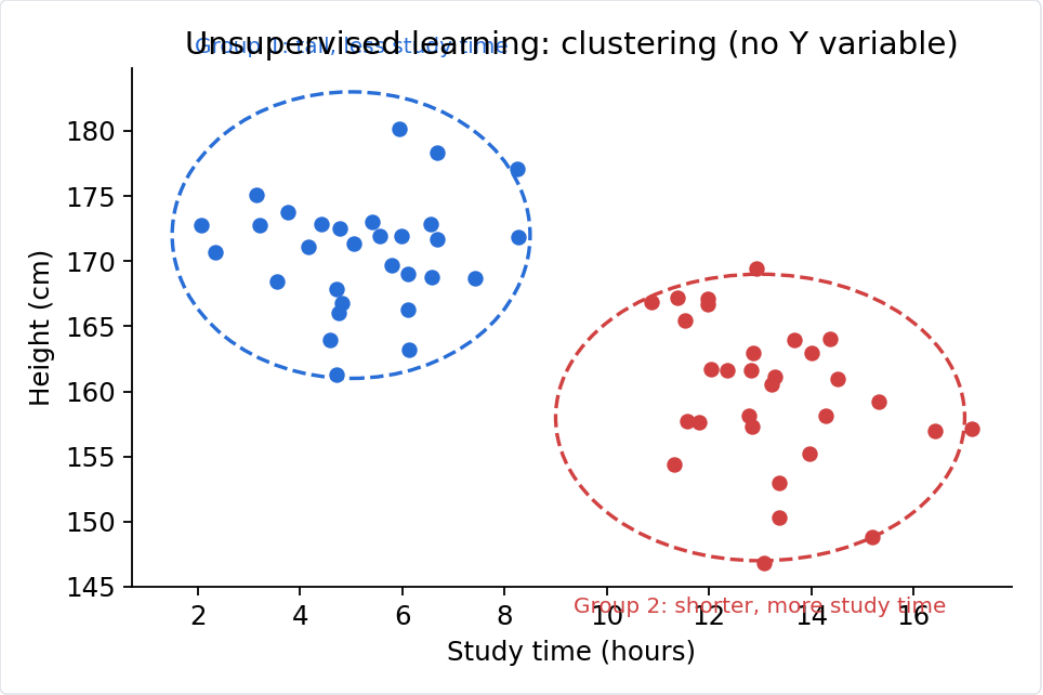
- The response is **qualitative (categorical)** with two categories → a **binary classification** problem.
- We still have a response variable, so it is **still supervised learning**.

> **Remember**
> Y quantitative (a number, e.g. marks, price) → **regression**. Y qualitative (a category, e.g. pass/fail) → **classification**. Both are **supervised**.

5. Unsupervised learning

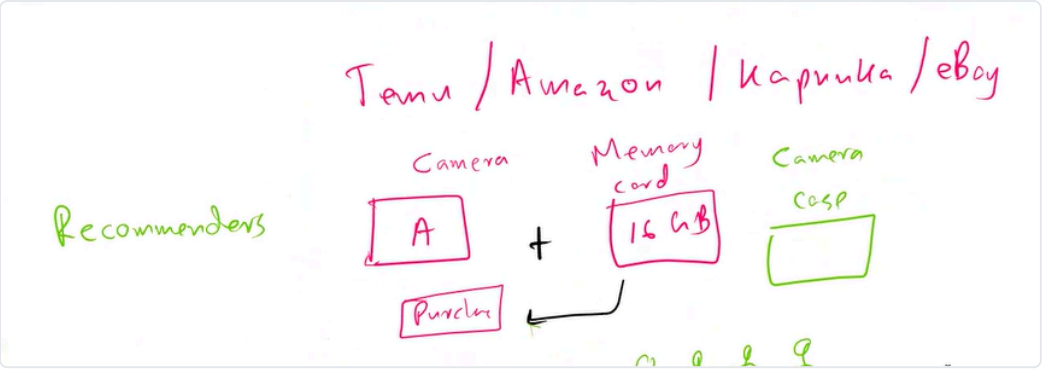
5.1 Clustering

Here there is **no response variable**. We only measure some characteristics, for example **study time** and **height**. Marks is not used as a response. When we plot them we may see **groups (clusters)** – for example one group is tall and studies less (maybe boys), another group is shorter and studies more (maybe girls). Finding such groups is **clustering**, which is **unsupervised learning**.



No Y variable – we find natural groups in the data

5.2 Recommender systems



Whiteboard: online shops recommend related products

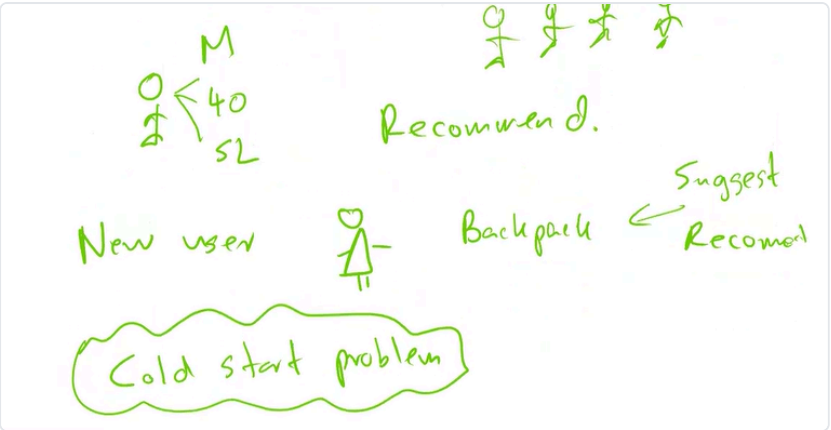
When you buy something from an online shop such as **Temu, Amazon, Kapruka or eBay**:

- You add a **camera (item A)** to the cart or purchase it.
- The site says "people who bought a camera also bought a **16 GB memory card**" and suggests it.
- Next time it may suggest a **camera case**.

This is a **recommender system**. The company may know about you (e.g. male, 40 years old, living in Sri Lanka), your **past purchases**, and what **other similar people** bought. It matches these and **recommends / suggests** items.

Netflix is another example: after you watch a movie (e.g. an X-Men movie), it suggests the next movie. Some years ago Netflix ran a public competition (the **Netflix Prize**) with a prize of about a million dollars for anyone who could improve their recommender system. You can search it online.

5.3 The cold start problem

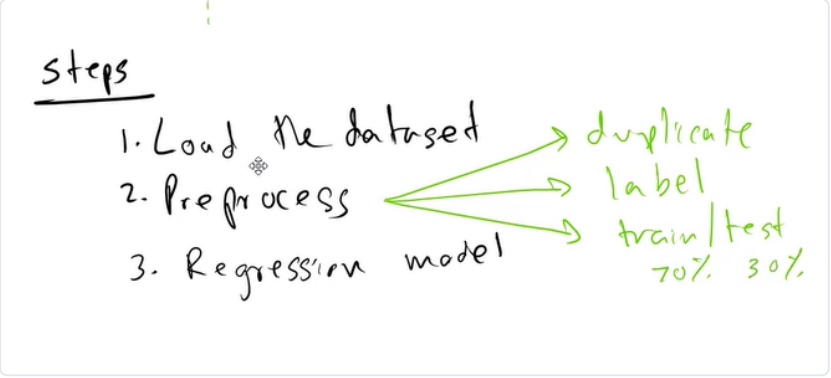


Whiteboard: a new user has no history

A **new user** joins and wants to buy a **backpack**. The company has **no past data** about this person, but it still needs to recommend something (shoes, another bag, …). How can it recommend without data? This difficulty is called the **cold start problem**. Researchers are still working on it.

6. Steps to solve a data problem

Example: the **Boston house** data set. You are given a CSV file with an ID, the house size in **square feet**, many other variables, and the **price** of the house. **Objective:** build a model to **predict the price**. Price is the **target variable**.



Whiteboard: the steps students suggested

Steps suggested by the class and completed by the lecturer:

1. **Load the data set** (the data is already given; no need to collect it).
2. **Pre-process the data**:
   - remove **duplicate** records
   - **label** the data if needed
   - handle **missing values**
   - handle **outliers**
   - **normalize** or **standardize** (rescale) the data
   - **dummy coding** for categorical variables
3. **Exploratory Data Analysis (EDA)** – plots and summaries.

4. **Split** into training and testing sets (70:30 or 80:20).
5. **Choose and fit a suitable model** (e.g. a regression model) on the training set.
6. **Test** the model on the test set and measure error, e.g. **MSE (mean squared error)**.

7. Data preprocessing

7.1 Dummy coding (categorical → numbers)

Models need numbers, so we convert a **categorical variable** into numeric form.

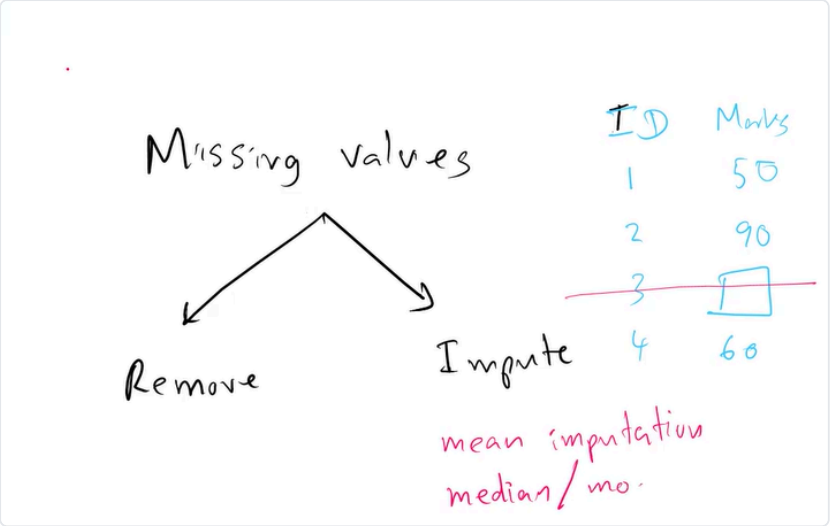
Gender: Male = 1, Female = 0.

Education level (4 levels):

Education level	Code
Postgraduate qualification	1
BSc	2
A/L	3
Less than A/L	0 (reference level)

In regression we create **indicator (dummy) variables** so that parameters can be estimated. One level is taken as the **reference level (0)** – some tools use the last level, some use the first level. Machine learning tools also need this kind of numeric coding.

7.2 Missing values



Whiteboard: remove or impute

There are **two options**:

1. **Remove** – delete the row that has the missing value (in R, e.g. `na.omit()`).
2. **Impute** – artificially fill the missing value with a sensible value.

ID	Marks
1	50
2	90
3	**missing**
4	60

- **Remove:** delete row 3.
- **Mean imputation:** mean = (50 + 90 + 60) / 3 = 200 / 3 = **66.67** → put 66.67 in row 3.
- **Median imputation:** use the median (better when there are outliers). Here median of 50, 60, 90 = 60.
- **Mode imputation:** for a **categorical** variable, use the most common category.

Rule of thumb (from the lecture)

- Missing < about 5–10% → you can **remove** those rows.
- Missing about 10–20% → **impute**.
- Missing more than about 50% → **remove the whole variable**. If most values are replaced, you are working with an artificial variable, so there is no point keeping it.

7.3 Outliers

An **outlier** is a value that is very far from the others. Looking at a plot ("eyeballing") can help, but it is not a reliable way to decide. A better way is the **IQR method** (the box plot uses this). In a box plot, outliers are shown as **circles or asterisks**.

$$\text{IQR} = \text{Q3} - \text{Q1}$$

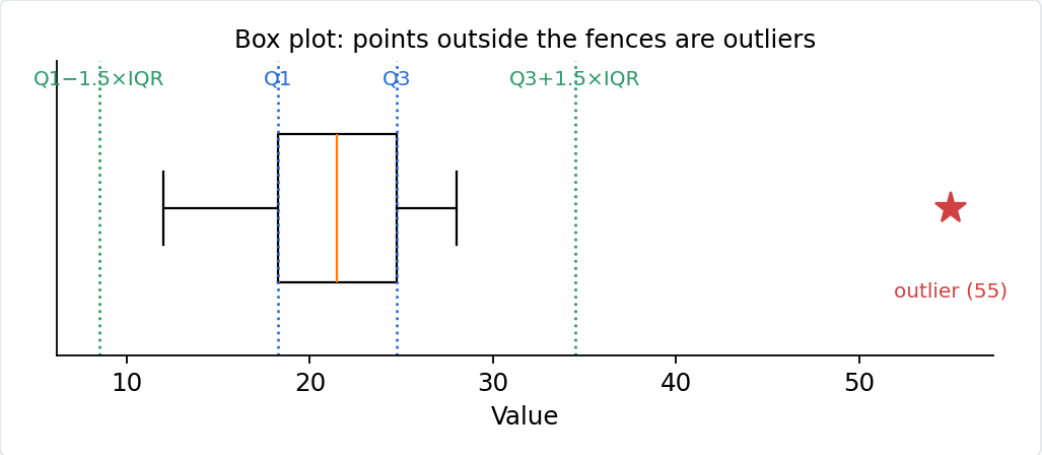
$$\text{Minimum usual value (lower fence)} = \text{Q1} - 1.5 \times \text{IQR}$$

$$\text{Maximum usual value (upper fence)} = \text{Q3} + 1.5 \times \text{IQR}$$

Any value **below the lower fence or above the upper fence** is an outlier.

Worked example, step by step

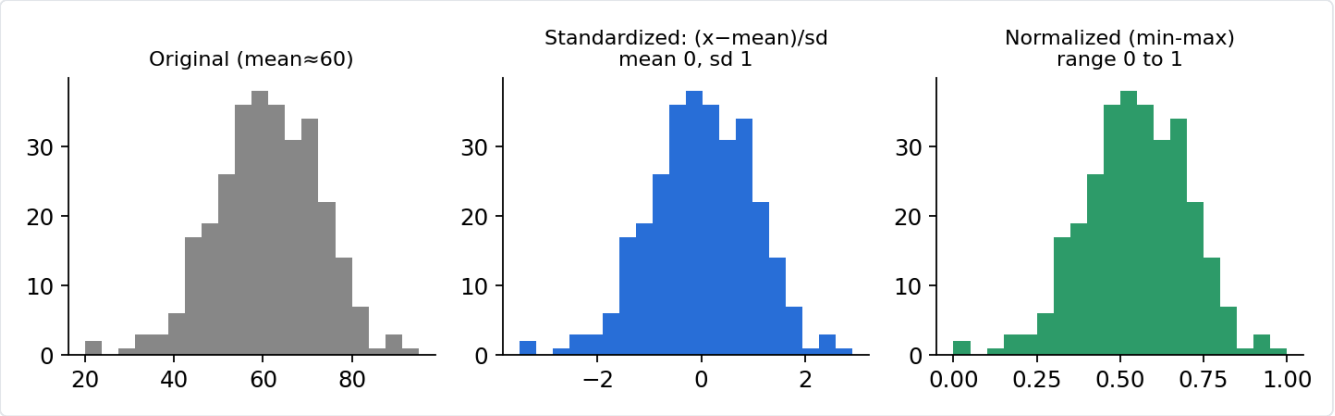
Data: 12, 15, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 28, 55
Step 1: Q1 = 18.25, Q3 = 24.75 (R: `quantile(x)`)
Step 2: IQR = 24.75 − 18.25 = 6.5
Step 3: 1.5 × IQR = 9.75
Step 4: Lower fence = 18.25 − 9.75 = 8.5; Upper fence = 24.75 + 9.75 = 34.5
Step 5: 55 > 34.5, so **55 is an outlier**. All other values are between 8.5 and 34.5.



7.4 Scaling: standardization and normalization

When variables are in **different units** (e.g. square feet and number of rooms), their spreads are very different, so we **rescale** them.

	Standardization	**Normalization (min-max)**
When to use	Data are roughly **normal**	Data are **not normal**
Formula	z = (x − mean) / standard deviation	x' = (x − min) / (max − min)
Result	mean 0, standard deviation 1	values between 0 and 1
Example	mean 60, sd 10, x = 75 → z = 15/10 = 1.5	min 20, max 100, x = 60 → 40/80 = 0.5

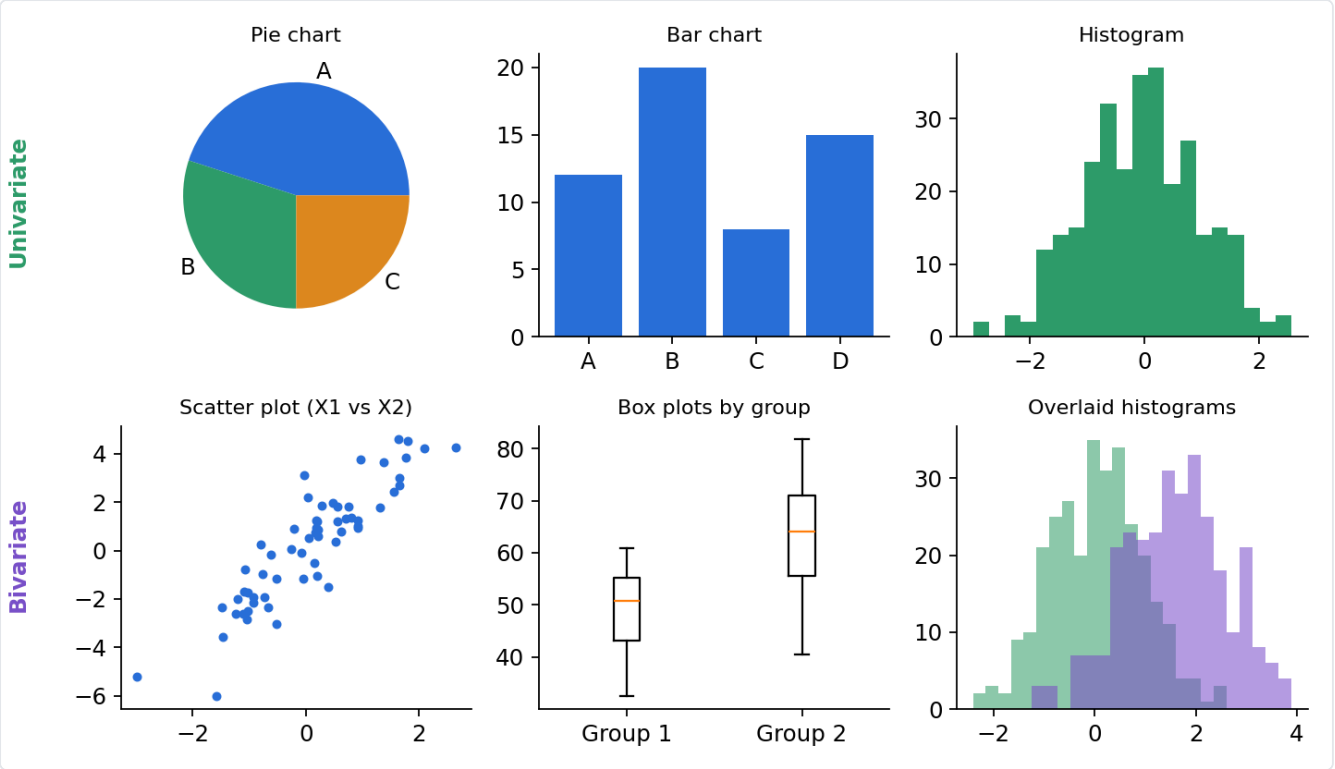


The shape stays the same; only the scale changes

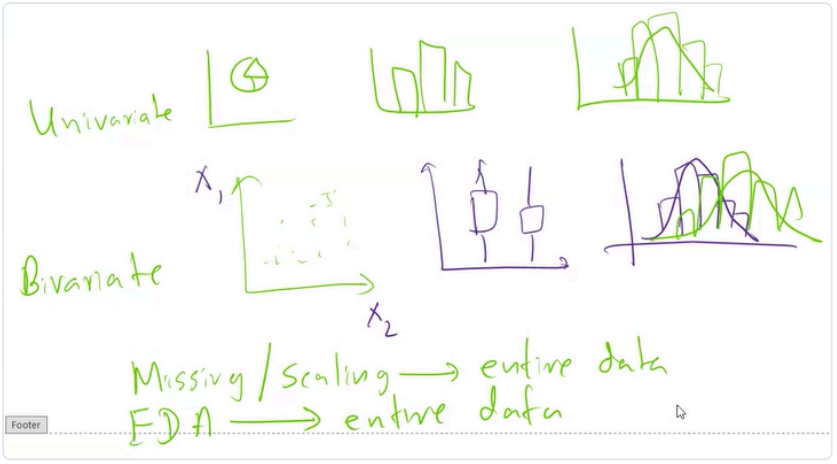
8. Exploratory Data Analysis (EDA)

EDA means looking at the data with plots and summaries to understand the variables before modelling.

- **Univariate** (one variable at a time): pie charts, bar charts, histograms.
- **Bivariate** (two variables together): scatter plots (X1 vs X2, or X vs the response), box plots by group, overlaid histograms.

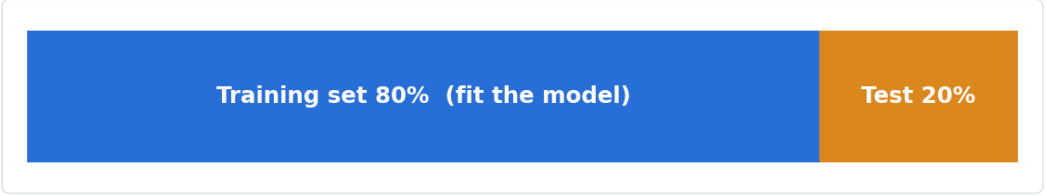


Common EDA plots



Whiteboard version

9. Train / test split

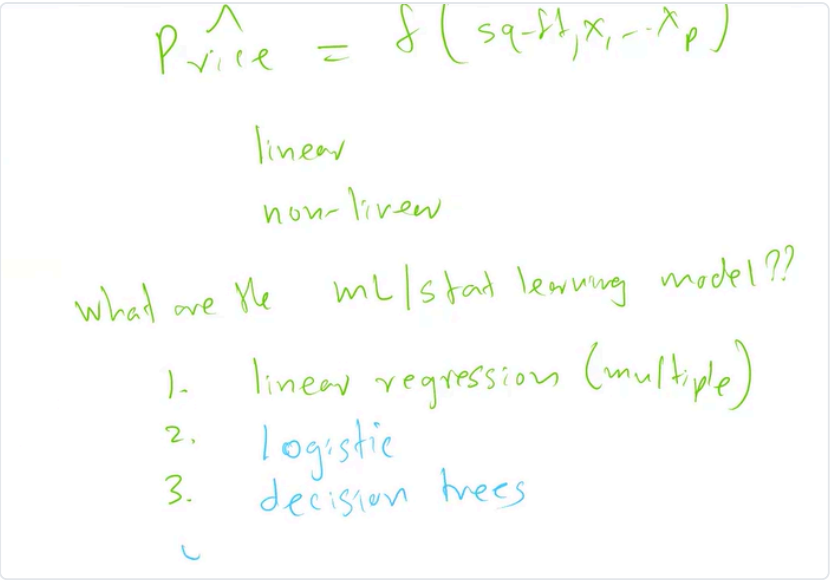


- Split the data into a **training set** (to fit the model) and a **test set** (to check how well it predicts new data). Usual ratios: **80:20** or **70:30**.
- **Two approaches:**
  1. Do missing value imputation, outlier detection, scaling and EDA on the **entire data set**, then split. After this, both train and test sets have no missing values.

2. Do scaling etc. **separately** for train and test. This is more involved: e.g. do you use the training mean and variance for the test set, or the overall mean? There are debates about this.

> **For this course**
> Do the missing values, scaling, outlier detection and EDA on the **entire data set**, **then** split into train and test, and train the model on the training set.

10. Models for a regression problem



Whiteboard: price as a function of the X variables

We want to model the price as a function of other variables:

$$\hat{\text{Price}} = f(\text{sq-ft}, X_1, \ldots, X_p)$$

The function f can be **linear** or **non-linear**. Machine / statistical learning models that can be used when Y is a number (price):

- Multiple linear regression
- Polynomial regression / other non-linear regression
- Decision (regression) trees
- Random forest
- Support vector regression (SVR)
- XGBoost
- KNN regression
- Ridge / Lasso regression (regularization)

> **Common mistake**
> Some students said **logistic regression** (it is also on the whiteboard). But price is a **quantitative** variable. Logistic regression and linear discriminant analysis (LDA) are for **classification**, so they **cannot** be used to predict price.

11. Important words

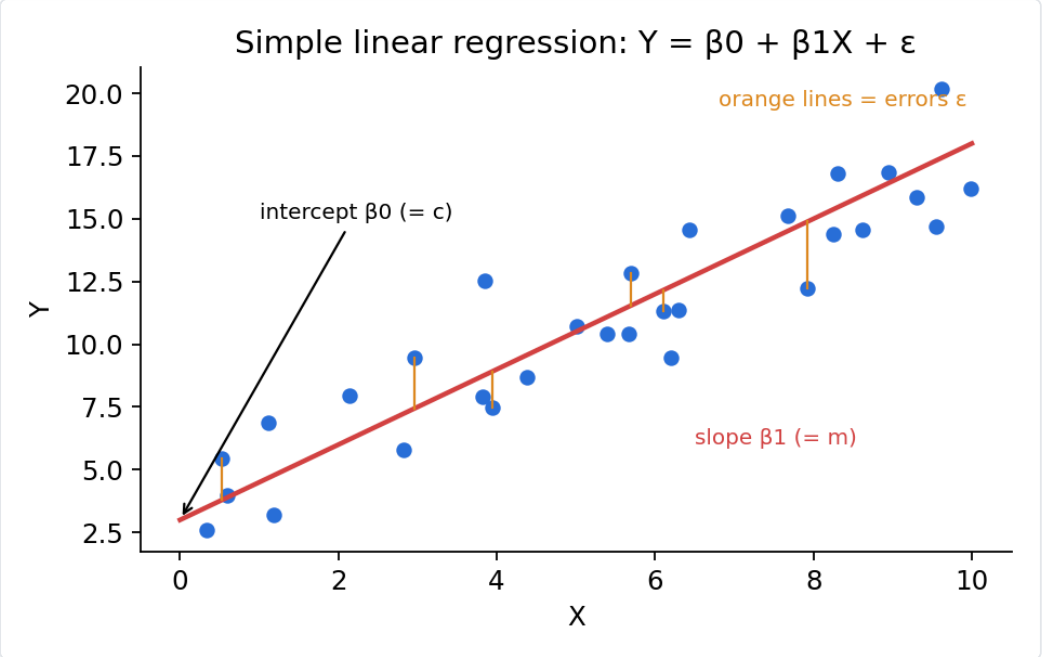
Word	Meaning
X variables	Predictors / independent variables / explanatory variables
Y variable	Target / response / dependent variable (sometimes called the "label")
Multicollinearity	When the **X variables are related (correlated) to each other**. We expect Y to be related to X1, X2, …, but sometimes X1, X2, … are also related among themselves. This causes problems in regression.

12. Linear regression, Ridge and Lasso

12.1 Simple linear regression

In mathematics a straight line is **y = mx + c** (c = intercept, m = slope). In statistics we use Greek letters and add an error term to handle points that are not on the line:

$$Y = \beta_0 + \beta_1 X + \varepsilon$$



12.2 Multiple linear regression

When there is more than one X variable:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p + \varepsilon$$

After fitting, the prediction is $\hat{Y} = \hat{\beta}_0 + \hat{\beta}_1 X_1 + \hat{\beta}_2 X_2 + \hat{\beta}_3 X_3 + \ldots$ (the error term goes away).

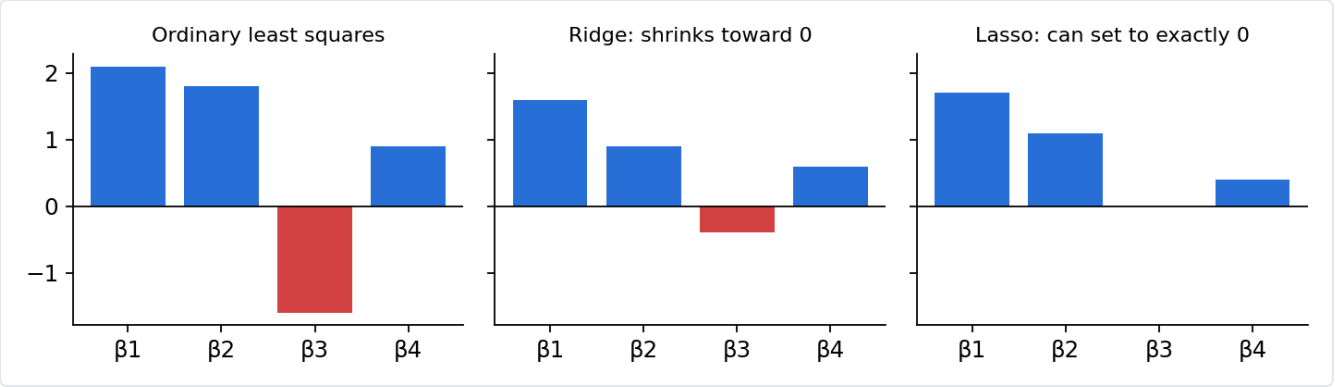
12.3 Dealing with multicollinearity – Ridge and Lasso

Suppose **X2 and X3 are related**. We do not need both – one of them can explain Y.

- **Ridge regression (L2)** tries to make the β estimates **as small as possible**. It shrinks one of the related coefficients (e.g. β_2) to a **small value** like 0.01, so X2's contribution to

Y becomes very small.

- **Lasso regression (L1)** can set a coefficient **exactly to zero** (e.g. $\beta_2 = 0$). Then X2 disappears from the model and there is no multicollinearity.

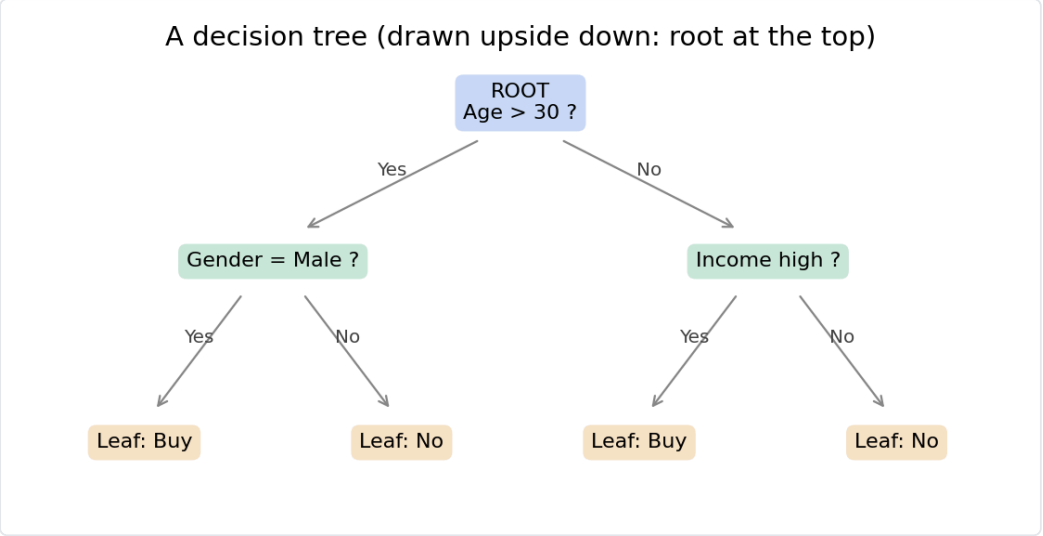


Ridge shrinks; Lasso can remove a variable completely

These methods are called **regularization techniques**. They can be used for **both regression and classification** – e.g. Ridge can be used with linear regression **and** with logistic regression.

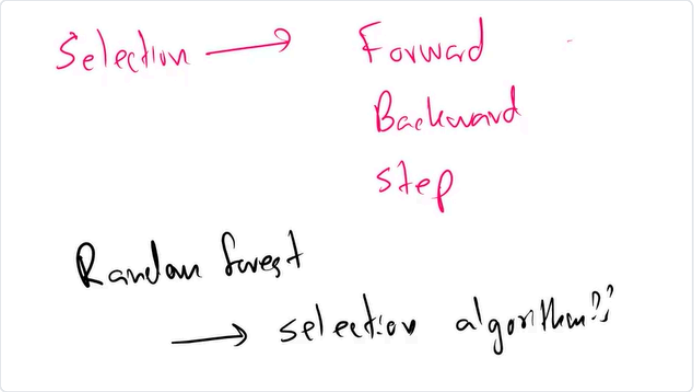
13. Decision trees and Random Forest

A normal tree has roots at the bottom. A **decision tree** is drawn **upside down**: the **root** is at the top and it splits into **branches**. At each split there is a **criterion** (condition), e.g. "Age > 30?" → go left or right; "Gender = male?" → left or right. This is a **tree-based model**.



- A bunch of trees = a **forest**.
- If there are, say, 100 variables (X1 … X100), we do not draw trees with every possible combination. At each tree/split we **randomly select** some variables. That is why it is called a **random forest**.
- Random forest can be used for **regression and classification**.

14. Variable selection

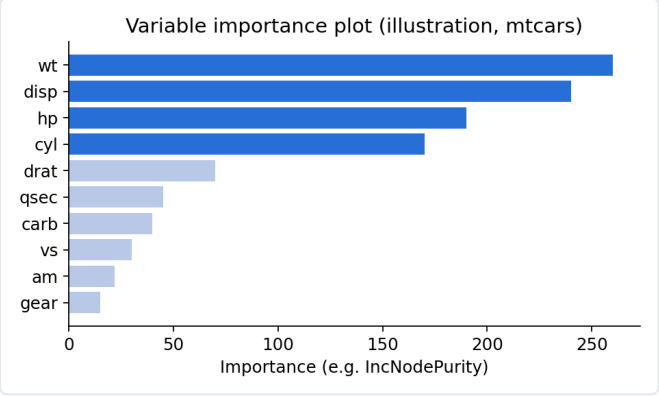


14.1 In multiple linear regression

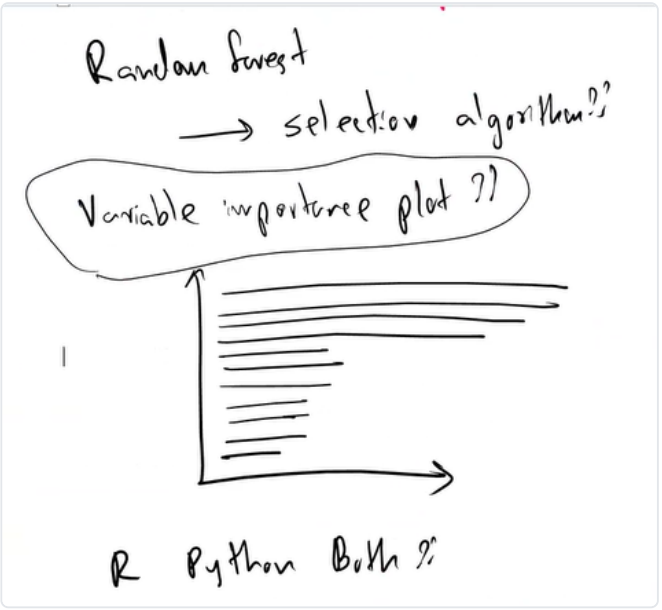
- **Forward selection** – start with no variables (null model) and add the most useful variable one by one.
- **Backward elimination** – start with all variables (full model) and remove the least useful one by one.
- **Stepwise** – a mix: variables can be added and removed at each step.

14.2 In random forest – the variable importance plot

Random forest gives a **variable importance plot**. Each variable has a horizontal bar: a **long bar = important**, a **short bar = not important**. We do not put all variables into the model – we choose the important ones.



Illustration

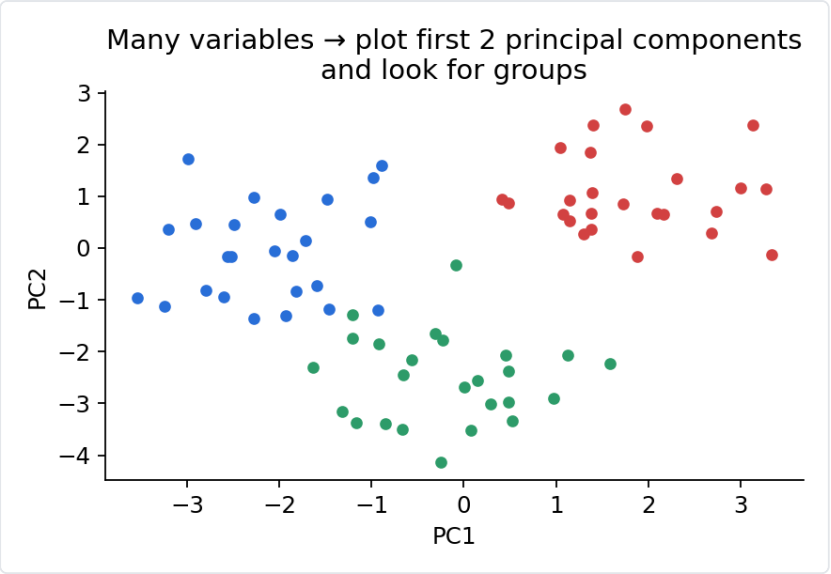


Whiteboard

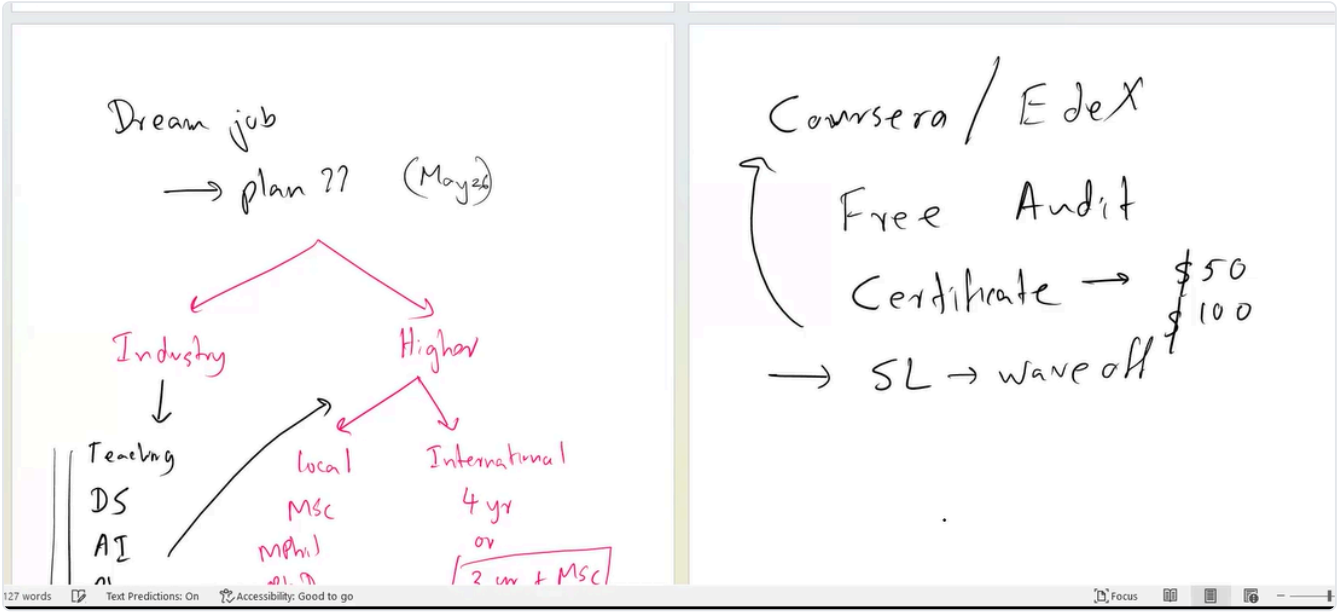
In R: `randomForest(..., importance = TRUE)` then `varImpPlot(model)`.

15. Dimension reduction – PCA

When there are **very many variables** (very common in biological applications), we cannot visualize them all. **Principal Component Analysis (PCA)** creates new variables called **principal components** that keep most of the information. We then plot the **first two principal components** and look for groupings. PCA is a **dimension reduction technique** (unsupervised).



16. Career advice from the lecturer



Whiteboard: plan after graduation

The class is a 3-year general degree (Applied Science) batch of about 300 students, of whom around 60 take statistics. The lecturer asked about your dream job (answers: teaching, higher studies, chemistry, AI engineer, …) and gave this advice.

Two paths after graduation (about May 2026)

- **Industry** – teaching, data science, AI, physics, chemistry, computer science jobs, etc.

- **Higher studies**
  - **Local:** MSc, MPhil, PhD – you are qualified, but there is competition.
  - **International:** usually needs a **4-year degree**, or a **3-year degree + MSc**.
- Some people work a few years and then do higher studies; some study first and then join industry.

How to stand out from other graduates

You compete with 300 batch-mates and 3-year general graduates from Colombo, Peradeniya, Jayewardenepura and other universities. Everyone's CV looks the same, so you must "sell yourself":

1. **Online courses** on **Coursera / edX** from top universities (Stanford, MIT, Harvard). You can **audit for free**. A certificate costs about $50–$100. Finish the course and assignments, then **politely email the organizers**, explain you are from Sri Lanka (a developing country) and ask them to **waive the fee**. With or without the certificate, add the course to your CV.
2. Create a **LinkedIn** page with a **professional photo** (not a social-media photo). Follow companies you like and build your network – they may notice you when there is a vacancy.
3. Keep an **updated CV**. Do not use the same common Canva template as everyone else.
4. Create a **GitHub** page and upload your projects and code so employers can see them.
5. Write **blogs on Medium** so employers can see how you write.

17. R practical – the mtcars example

Tasks given in class: (1) load the `mtcars` data set; (2) how many variables? (3) how many records? (4) any missing values? (5) any outliers? (6) fit a suitable model to predict **mpg** (miles per gallon), using seed **123** to split the data **80:20** into train and test sets.

> **Lecturer's advice**
> Do not ask AI tools to write this code for you. Try to write your own code – in exams and assignments you must write it yourself.

Step 1 – Load the data

```
# STA 3402 – Day 1
# 1. load the mtcars dataset
data(mtcars)
```

`mtcars` is a built-in R data set about 32 cars (from a 1974 magazine). Type `?mtcars` to read the help page. Main variables: **mpg** (fuel efficiency), cyl (cylinders), disp (engine size), hp (horsepower), drat, **wt** (weight, 1000 lbs), qsec (1/4 mile time), vs, am (transmission), gear, carb.

Step 2 & 3 – Number of variables and records

```
# 2. how many variables are there?
dim(mtcars)     # gives rows and columns: 32 11
ncol(mtcars)    # number of columns (variables) = 11

# 3. how many records are there?
nrow(mtcars)    # number of rows (records) = 32
```

Answer: **11 variables** and **32 records**.

Step 4 – Missing values

```
A = c(10,15,20,25,34)
A
is.na(A)          # FALSE FALSE FALSE FALSE FALSE  (no missing)

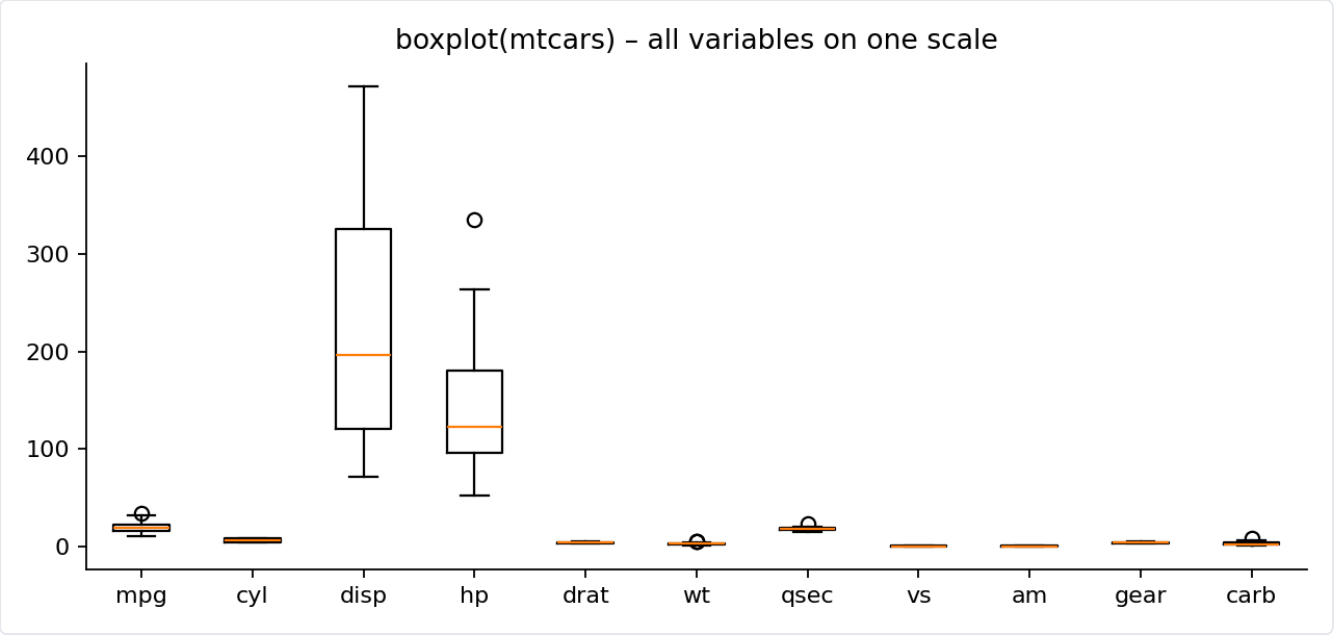
B = c(10,15,NA,34,85)
B
is.na(B)          # FALSE FALSE TRUE FALSE FALSE
sum(is.na(B))     # 1  (TRUE counts as 1, so this counts missing values)

summary(mtcars)   # summary of each variable; missing values would show as NA's
sum(is.na(mtcars))# 0  -> no missing values in mtcars
```

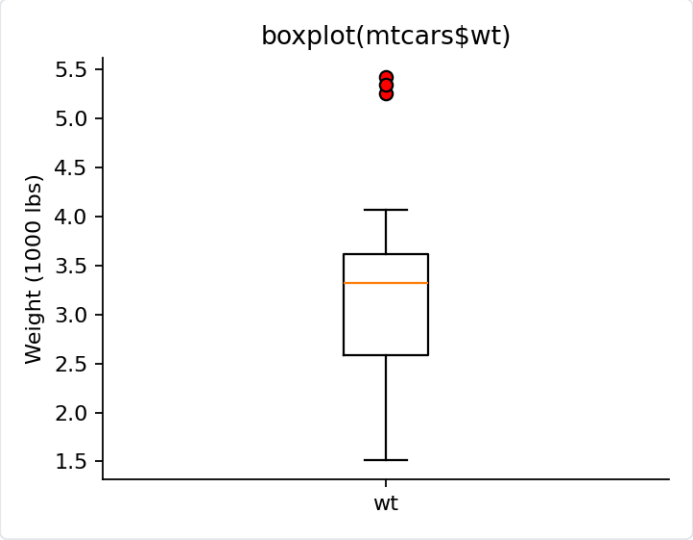
How it works: `is.na()` returns TRUE for each missing value. `sum()` counts the TRUEs. Other options from the class: `anyNA(mtcars)` (TRUE/FALSE – any missing at all?) and `any(is.na(mtcars))`. These are all in base R, so no package is needed. Answer: **no missing values**.

Step 5 – Outliers

```
# 5. are there any outliers?
boxplot(mtcars)             # box plot for all variables
boxplot.stats(mtcars$mpg)   # $out is empty -> no outliers in mpg
boxplot.stats(mtcars$wt)    # $out = 5.424 5.345 -> two outliers
boxplot(mtcars$wt)
```



boxplot(mtcars) – disp and hp are on a much bigger scale, so other boxes look flat (another reason to scale)



wt has two outliers (5.424 and 5.345)

`boxplot.stats()` gives the box plot numbers. In `$stats` for wt: 1.513, 2.5425, 3.325, 3.65, 5.25 (lower whisker, lower hinge, median, upper hinge, upper whisker). `$out` lists the outliers. For mpg, `$out` is `numeric(0)`, meaning **no outliers**. You should check each variable this way.

Step 6 – Train / test split with a seed

```
# 6. fit a suitable prediction model to predict mpg
# use 123 as the seed to split 80:20 train and test sets
set.seed(123)
trainID = sample(1:nrow(mtcars), 0.8*nrow(mtcars), replace = FALSE)
trainSet = mtcars[trainID,]
head(trainSet)

testID = mtcars[-trainID,]    # this is actually the TEST SET
head(testID)
```

Line by line:

- `1:nrow(mtcars)` = row numbers 1 to 32. Using `nrow()` instead of typing 32 makes the code general. (In class the lecturer first typed `ncol` by mistake – a student pointed out it must be **nrow**, the number of rows.)
- `0.8*nrow(mtcars)` = 25.6 → R takes **25 rows** for training; the other **7 rows** are the test set.
- `replace = FALSE` – a row cannot be chosen twice.
- `mtcars[trainID, ]` – a data frame is indexed as `[row, column]`. We give the chosen row numbers and leave the column part empty, so **all columns** are taken.
- `mtcars[-trainID, ]` – the minus sign means "all rows **except** these" → the test set.
- `head()` shows the first 6 rows so you can take a quick look.

> **Why set.seed()?**
> `sample()` picks rows at random, so each time you run it you get a different train and test set. If you write half your report today and rerun the code tomorrow, the results change and you must rewrite the report.
> A computer cannot "think" of a random number like a person can (when asked, a student just said "5"). It uses an **algorithm** that takes an input and produces numbers – these are **pseudo-random numbers**. If you do not give a seed, R uses the **clock time** as the input, so results change every time. If you set the seed (123), the input is fixed, so you get the **same sample every time** → results can be **reproduced**.

Step 7 – Fit a multiple linear regression

```
?mtcars
# mpg is a continuous variable -> regression problem

# multiple linear regression
mlrOutput = lm(trainSet$mpg ~ ., data = trainSet)
# dot means use all the variables
summary(mlrOutput)
```

- `lm()` = linear model. The left of `~` is the response, the right side is the predictors. The **dot (.)** means "all other variables in the data".
- `summary()` shows the intercept, each coefficient ($\hat{\beta}$), its p-value, R^2, etc. In class, the full model had the intercept around 21, **cyl had a negative coefficient**, and only one variable was significant, and only at the **10%** level. So we need **variable selection**.

> **Useful tip (important!)**
> Write the formula as `lm(mpg ~ ., data = trainSet)` rather than `lm(trainSet$mpg ~ ., data = trainSet)`. With `trainSet$mpg ~ .`, R does not recognise the left side as the column "mpg", so the dot can include **mpg itself as a predictor** – this gives a "perfect fit" warning and meaningless results. Using just the column name avoids this. The same applies to `nullModel`, `fullModel` below.

Step 8 – Variable selection with step()

```
# null model (only intercept)
nullModel = lm(trainSet$mpg ~ 1, data=trainSet)
## step(nullModel, direction = "forward", scope=c(list all the variables), data=trainSet)

# full model (all variables)
fullModel = lm(trainSet$mpg ~ ., data = trainSet)
step(fullModel, direction = "backward", data=trainSet)

selModel = step(fullModel, direction = "backward", data=trainSet)
summary(selModel)
```

- **Null model** (`~ 1`) = only the intercept, no predictors. Used as the starting point for **forward** selection. You must give the `scope` (the list of variables it is allowed to add), e.g. `scope = formula(fullModel)`.
- **Full model** = all variables. Used as the starting point for **backward** elimination.
- `step()` uses **AIC** (smaller is better). In each iteration it removes the variable whose removal lowers AIC most, and stops when no removal helps. In class, the final model kept **cyl, wt and qsec**.
- `summary(selModel)` showed that some of these were not significant, and **wt** was the clearly significant one. So the lecturer fitted the final model with **only wt**:

```
mFinal = lm(mpg ~ wt, data = trainSet)
summary(mFinal)
```

Meaning: mpg depends mainly on the **weight** of the car – heavier cars give **fewer miles per gallon** (negative slope).

Step 9 – Random forest

```
library(randomForest)          # install.packages("randomForest") the first time
set.seed(123)
rf_model = randomForest(mpg ~ ., data = trainSet)
rf_model
```

Printing `rf_model` shows the type (regression), the number of trees (500 by default), the number of variables tried at each split, the mean of squared residuals and the "% Var explained" (like R^2). Add `importance = TRUE` and use `varImpPlot(rf_model)` to see the variable importance plot.

Step 10 – Checking the model on the test set (to complete the workflow)

```
pred_lm = predict(mFinal, newdata = testID)
mean((testID$mpg - pred_lm)^2)      # test MSE for linear model

pred_rf = predict(rf_model, newdata = testID)
mean((testID$mpg - pred_rf)^2)      # test MSE for random forest
```

The model with the **smaller test MSE** predicts better on new data.

To try next (listed in the code)
XGBoost, Support Vector Regression (SVR) and regularization (Ridge/Lasso). The lecture stopped here; these continue in the next session.

18. Assignment 1 – Kaggle competition

Item	Details
Deadline	One week – **next Monday by 5.00 pm**
Data	A challenging data set on the **mental health condition of patients related to drug usage**
Task	Build a model to predict the **drug category** → a **classification** problem
Platform	Set up as a **Kaggle competition** (link in Google Drive / WhatsApp)

How to do it

1. Create a Kaggle account with your Gmail if you do not have one. Your **screen name must be your real name** (first name, last name) so you can be identified.
2. Log in, open the given link and click **"Join Competition"**.
3. In the Data tab you will find: **train** data set, **test** data set, **sample submission** file and a **variable description**.
4. The **training** set has the response variable (drug category). The **test** set does **not**.
5. Fit your classification model on the training data and predict the drug category for the test data.
6. Put your predictions in the **same format as the sample submission** file (you can download it and paste your predictions into the second column).
7. Click **Submit Predictions**. Your score (accuracy) will appear on the **leaderboard** with your name, ranked against your friends.
8. You can make **5 submissions per day** (resets at 12 midnight) – about 35 submissions in a week. Try to improve your score!

The report (PDF)

- Upload a PDF (upload link will be shared – a Google Form until LMS access is available).
- It must contain: **data pre-processing steps**; any **exploratory data analysis**; **model building steps** clearly and thoroughly; your **final model** in a clear, well-organized way.
- Marks are deducted for missing or incomplete steps – giving only the final model is not enough.
- Format: **no more than 8 pages**, **Times New Roman, size 12**, **1.5 line spacing**.
- Your best Kaggle model becomes the final model you report.

19. Summary and revision questions

Key points

- DL ⊂ ML ⊂ AI. ML = supervised (has Y) + unsupervised (no Y).
- Y numeric → regression; Y category → classification.
- Workflow: load → pre-process (duplicates, dummy coding, missing, outliers, scaling) → EDA → split → model → test (MSE).
- Missing values: remove or impute (mean / median / mode).
- Outliers: IQR rule, Q1 − 1.5·IQR and Q3 + 1.5·IQR.
- Standardize if normal; normalize (min-max) if not.
- Multicollinearity → Ridge (shrink) or Lasso (set to 0) = regularization.
- Variable selection: forward, backward, stepwise; RF → variable importance plot.
- Always `set.seed()` for reproducible results.

Test yourself

1. Give one example each of supervised and unsupervised learning.
2. Why is predicting pass/fail still supervised learning?
3. What is the cold start problem? Give an example.
4. Marks: 40, 70, NA, 55. Find the mean-imputed value. (Answer: 55)
5. Q1 = 20, Q3 = 30. Find the outlier fences. (Answer: 5 and 45)
6. When do you standardize and when do you normalize?
7. Why can't logistic regression be used to predict house price?
8. What is multicollinearity, and how do Ridge and Lasso deal with it differently?
9. Why is a random forest called "random"?
10. What does `set.seed(123)` do and why do we need it?
11. In R, what does the dot in `lm(mpg ~ ., data = trainSet)` mean?
12. What does `mtcars[-trainID, ]` return?